

Algoritmica Grafurilor - Cursul 9

4 decembrie 2020

1

Fluxuri în rețele

- Prefluxuri

 - Schema generală a unui algoritm de tip preflux

 - Algoritmul lui Ahuja & Orlin

- Aplicații combinatoriale

 - Cuplaje în grafuri bipartite

 - Recunoașterea secvențelor digrafice

 - Conexiunea pe muchii

 - Conexiunea pe noduri

2

Exerciții pentru seminarul din săptămâna viitoare

3

Exerciții rezolvate (parțial)

Definiție

Un preflux în $R = (G, s, t, c)$ este o funcție $x : E(G) \rightarrow \mathbb{R}$ astfel încât

- (i) $0 \leq x_{ij} \leq c_{ij}, \forall ij \in E;$
- (ii) $\forall i \neq s, e_i = \sum_{ji \in E} x_{ji} - \sum_{ij \in E} x_{ij} \geq 0.$

e_i (pentru $i \in V \setminus \{s, t\}$) este numit **excesul** din nodul i . Dacă $i \in V \setminus \{s, t\}$ și $e_i > 0$, atunci i este un **nod activ**. Dacă $ij \in E$, x_{ij} va fi numit **fluxul de pe arcul** ij .

Dacă în R nu există noduri active, atunci prefluxul x este un flux cu $v(x) = e_t$.

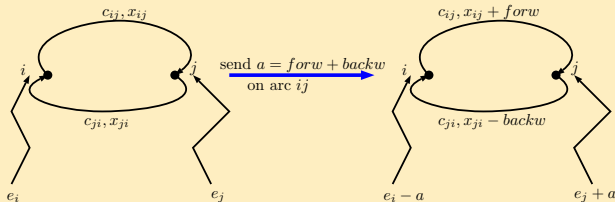
Ideea algoritmilor de tip preflux: un preflux inițial în R este transformat prin modificarea fluxurilor de pe arce într-un flux cu proprietatea că nu există drumuri de creștere în R relativ la el.

G este reprezentat folosind liste de adiacență. Vom presupune că dacă $ij \in E$, atunci $ji \in E$ de asemeni (altfel, adăugăm arcul ji de capacitate 0). Astfel, G devine digraf simetric.

Dacă x este un preflux în R și $ij \in E$, atunci **capacitatea reziduală** a lui ij este

$$r_{ij} = c_{ij} - x_{ij} + x_{ji},$$

(reprezentând fluxul suplimentar care poate fi trimis de la nodul i la nodul j folosind arcele ij și ji).



În cele de urmează, a trimite flux de la i la j înseamnă creșterea fluxului pe arcul ij sau scăderea fluxului pe arcul ji .

Un A -drum în R relativ la prefluxul x , este un drum în G având toate arcele de capacitate reziduală strict pozitivă.

O funcție distanță în R relativ la prefluxul x este o funcție $d : V \rightarrow \mathbb{Z}_+$ a. î.

$$(D_1) \quad d(t) = 0,$$

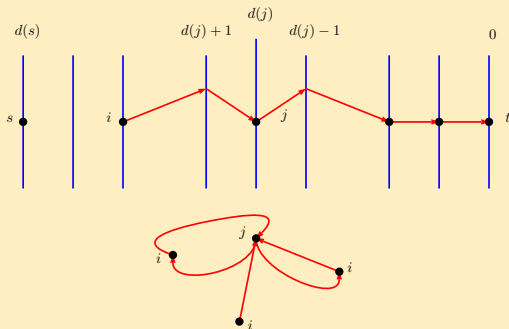
$$(D_2) \quad \forall ij \in E, r_{ij} > 0 \Rightarrow d(i) \leq d(j) + 1.$$

Remarci

- Dacă P este un A -drum relativ la prefluxul x în R de la i la t , atunci $d(i) \leq \text{length}(P)$ (arcele lui P au capacități reziduale pozitive și apoi se aplică (D_2) în mod repetat). Urmează că (τ_i) este lungimea minimă a unui A -drum de la i la t : $d(i) \leq \tau_i$.

Remarci

- Ca de obicei, notăm cu $A(i)$ lista de adiacență a nodului i .



Definiție

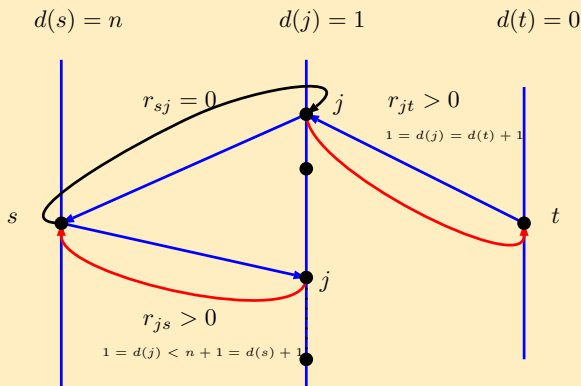
Fie x un preflux în R și d o funcție distanță relativ la x . Un arc $ij \in E$ este numit **admisibil** dacă $r_{ij} > 0$ și $d(i) = d(j) + 1$.

Dacă R este o rețea, considerăm următoarea procedură de inițializare, care construiește în $\mathcal{O}(m)$ un preflux x și o funcție distanță d relativ la x .

```
procedure initialization();  
for ( $ij \in E$ ) do  
    if ( $i = s$ ) then  
         $x_{sj} \leftarrow c_{sj}$   
    else  
         $x_{ij} \leftarrow 0$ ;  
 $d[s] \leftarrow n$ ;  $d[t] \leftarrow 0$ ;  
for ( $i \in V - \{s, t\}$ ) do  
     $d[i] \leftarrow 1$ ;
```

Se observă că după execuția acestei proceduri, avem $r_{sj} = 0, \forall sj \in A(s)$. Astfel condiția (D_2) nu este not afectată de $d(s) = n$.

Pentru toate arcele ij , (D_2) este îndeplinită:



Alegerea $d(s) = n$ are semnificația: nu există A -drumuri de la s la t relativ la x (altfel, dacă P este un astfel de drum, lungimea lui trebuie să fie cel puțin $d(s) = n$, ceea ce este imposibil).

Dacă acesta va fi un invariant al algoritmilor de tip preflux, atunci când x va deveni un flux, va fi un flux maxim.

Considerăm următoarele două proceduri:

procedure *push*(i); // i este un nod diferit de s, t

alege un arc admisibil $ij \in A(i)$;

"trimite" $\delta = \min \{e_i, r_{ij}\}$ (unități de flux) de la i la j ;

Dacă $\delta = r_{ij}$ atunci avem o **push/pompare saturată**, altfel avem o **pompare nesaturată**.

procedure *relabel*(i); // i este un nod diferit de s, t

$d[i] \leftarrow \min \{d[j] + 1 : ij \in A(i) \text{ și } r_{ij} > 0\}$;

Prefluxuri - Schema generală a unui algoritm de tip preflux

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

initialization;

while ($\exists$ noduri active în R) **do**

 alege un nod activ i ;

if ($\exists$ arce admisibile în $A(i)$) **then**

push(i);

else

relabel(i);

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Lema 1

" d este funcție distanță relativ la prefluxul x " este un invariant al algoritmului de mai sus. La fiecare apel *relabel*(i), $d(i)$ crește.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Demonstrație. Am demonstrat deja că procedura de inițializare construiește un preflux x și o funcție distanță d relativ la x . Arătăm că dacă perechea (d, x) satisface (D_1) și (D_2) înaintea unei iterații **while**, atunci după această iterație cele două condiții sunt de asemeni satisfăcute.

Avem două cazuri, în funcție de care dintre procedurile **push** sau **relabel** este executată în iterația **while** curentă:

este apelată $push(i)$: singura pereche care poate viola (D_2) este $d(i)$ și $d(j)$. Deoarece ij este admisibil, la apelul $push(i)$ avem $d(i) = d(j) + 1$. După execuția $push(i)$, arcul ji poate avea $r_{ji} > 0$ (fără a fi astfel înaintea apelului), dar condiția $d(j) \leq d(i) + 1$ este evident satisfăcută.

este apelată $relabel(i)$: actualizarea lui $d(i)$ este astfel încât (D_2) este satisfăcută pentru fiecare arc ij cu $r_{ij} > 0$. Deoarece $relabel(i)$ este apelată când $d(i) < d(j) + 1$, $\forall ij$ cu $r_{ij} > 0$, urmează că, după apel, $d(i)$ crește (cu cel puțin 1). $\square$

Pentru a arăta că algoritmul se oprește, este necesar să arătăm că (în timpul execuției) dacă un nod i este activ atunci în lista lui de adiacență, $A(i)$, există cel puțin un arc ij cu $r_{ij} > 0$. Aceasta demonstrează lema următoare.

Lema 2

Dacă i_0 este un x -nod activ în R , atunci există un i_0 's A -drum relativ la x .

Demonstrație. Dacă x este un preflux în R , atunci x poate fi descompus în $x = x^1 + x^2 + \dots + x^p$, unde fiecare x^k are proprietatea că mulțimea $A^k = \{ij : ij \in E, x_{ij}^k \neq 0\}$ este

- (a) mulțimea arcelor unui drum de la s la t , sau
- (b) mulțimea de arce ale unui drum de la s la un nod activ, sau
- (c) mulțimea arcelor unui circuit.

Prefluxuri - Schema generală a unui algoritm de tip preflux

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

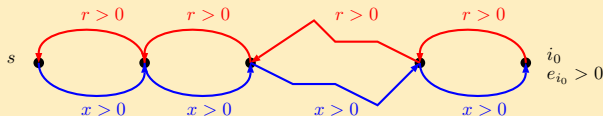
Mai mult, în cazurile (a) și (c), x^k este un flux. (Demonstrația este algoritmică: construim mai întâi mulțimile de tipul (a), apoi pe cele de tipul (c) și (b); la fiecare pas, căutăm inversul unui drum de tip (a) sau (c) (sau (b)); prefluxul obținut este scăzut din cel curent; deoarece excelele nodurilor sunt nenegative, construcția poate fi realizată, ori de câte ori prefluxul curent este nenul; construcția este finită, deoarece numărul de arce pe care fluxul curent este 0, crește la fiecare pas.)

Deoarece i_0 este un nod activ în R relativ la x , cazul (b) va apărea pentru nodul i_0 (cazurile (a) și (c) nu afectează excesul din nodul i_0). Arcele inverse de pe acest drum au capacitatea reziduală pozitivă (vezi imaginea de mai jos), astfel ele formează A -drumul cerut. $\square$

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Prefluxuri - Schema generală a unui algoritm de tip preflux

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph



Corolarul 1

$\forall i \in V, d(i) < 2n.$

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Demonstrație. Într-adevăr, dacă i nu a fost reetichetat, atunci $d(i) = 1 < 2n$. Altfel, înaintea apelului $relabel(i)$, i este un nod activ, astfel din Lema 2, există un is -drum P cu $length(P) \leq n - 1$. Din (D_2) , urmează că, după reetichetare, $d(i) \leq d(s) + n - 1 = 2n - 1$ ($d(s) = n$ nu este modificat vreodată). $\square$

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Corolarul 2

Numărul total de apeluri ale procedurii relabel nu este mai mare decât $2n^2$.

Demonstrație. Într-adevăr, există $n - 2$ noduri care pot fi reetichetate. Fiecare dintre ele poate fi reetichetat de cel mult $2n - 1$ ori (din Lema 1, Corolarul de mai sus și distanța inițială d). $\square$

Corolarul 3

Numărul total de pompări saturate nu este mai mare decât nm .

Demonstrație. Într-adevăr, când un arc ij devine saturat, avem $d(i) = d(j) + 1$. După aceea, algoritmul nu va mai trimite flux pe acest arc până când nu va trimite flux pe arcul ji , când vom avea $d'(j) = d'(i) + 1 \geq d(i) + 1 = d(j) + 2$.

Prefluxuri - Schema generală a unui algoritm de tip preflux

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Demonstrație (continuare). Astfel o modificare a acestui flux pe arcul ij nu va apărea până când $d(j)$ nu va crește cu 2. Urmează că un arc nu poate deveni saturat de mai mult de n ori și (deoarece numărul total de arce este m), nu există mai mult de nm pompări saturate. $\square$

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Lema 3

(Goldberg și Tarjan, 1986). Numărul total de pompări nesaturate nu este mai mare decât $2n^2m$.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Demonstrație. Omisă.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Lema 4

Algoritmul de tip preflux returnează un flux de valoare maximă în R .

Prefluxuri - Schema generală a unui algoritm de tip preflux

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Demonstrație. Din Lemele 1 și 3 și din Corolarul 3 al Lemei 2, algoritmul se oprește în cel mult $2n^2m$ iterații **while**. Deoarece $d(s) = n$ nu se modifică niciodată, urmează că nu există vreun drum de creștere relativ la fluxul x obținut, astfel x este de valoare maximă: **dacă P ar fi un drum de creștere (în digraful suport al lui G), atunci înlocuind de-a lungul lui P fiecare arc invers cu simetricul său se obține un A -drum de la s la t , deci $n = d(s) \leq d(t) + n - 1 = n - 1$. $\square$**

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

În locul demonstrației Lemei 3, vom prezenta **Algoritmul lui Ahuja & Orlin (1988)** care utilizează o **metodă de scalare - scaling method** pentru a reduce numărul de pompări nesaturate de la $\mathcal{O}(n^2m)$ (**Goldberg & Tarjan algorithm**) la $\mathcal{O}(n^2 \log U)$.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Să presupunem că $c_{ij} \in \mathbb{N}$ și $U = \max_{ij \in E} (1 + c_{ij})$. Fie $K = \lceil \log_2 U \rceil$.

Ideea algoritmului: Avem $K + 1$ etape. În fiecare etapă p , cu p luând succesiv valorile $K, K - 1, \dots, 1, 0$, următoarele două condiții sunt îndeplinite:

- (a) la începutul etapei p , $e_i \leq 2^p, \forall i \in V \setminus \{s, t\}$.
- (b) în timpul etapei p , procedurile push-relabel sunt folosite pentru a elimina nodurile active, i , cu $e_i \in \{2^{p-1} + 1, \dots, 2^p\}$.

Din definiția lui K , în prima etapă ($p = K$), condiția (a) este îndeplinită, și dacă condiția (b) va fi menținută de-a lungul execuției algoritmului, după $K + 1$ etape, dacă se menține integralitatea exceselor de-a lungul algoritmului, atunci urmează că, după etapa $K + 1$, **excesul fiecărui nod $i \in V \setminus \{s, t\}$ este 0, deci avem un flux de valoare maximă.**

Pentru a menține (b) de-a lungul execuției algoritmului, schema generală a unui algoritm de tip preflux este adaptată după cum urmează:

- fiecare etapă p începe prin construirea listei $L(p)$ a tuturor nodurilor $i_1, i_2, \dots, i_{l(p)}$ cu excese $e_i > 2^{p-1}$, ordonate crescător după d (aceasta poate fi făcută cu hash-sorting în timpul $O(n)$, deoarece $d(i) \in \{1, 2, \dots, 2n - 1\}$).
- nodul activ selectat pentru **push-relabel** în timpul etapei p va fi **primul nod din $L(p)$** . Urmează că, dacă se face o pompă (push) pe un arc admisibil ij , atunci $e_i > 2^{p-1}$ și $e_j \leq 2^{p-1}$ (deoarece $d(j) = d(i) - 1$ și i este primul nod din $L(p)$). Dacă δ , fluxul trimis de la i la j de către apelul $push(i)$, este limitat la $\delta = \min(e_i, r_{ij}, 2^p - e_j)$, atunci (deoarece $2^p - e_j \geq 2^{p-1}$) urmează că o **pompă nesaturată** trimite cel puțin 2^{p-1} unități de flux.

După apelul $push(i)$ excesul din nodul j (singurul pentru care excesul poate crește) va fi $e_j + \min(e_i, r_{ij}, 2^p - e_j) \leq e_j + 2^p - e_j \leq 2^p$, deci condiția (b) rămâne îndeplinită.

- etapa p este încheiată când lista $L(p)$ devine vidă.

Pentru a determina în mod eficient un arc admisibil pentru o pompă ($push$), sau pentru a inspecta toate arcele care părăsesc un nod i în vederea reetichetării ($relabel$), vom organiza listele de adiacență $A(i)$ după cum urmează:

- fiecare element din listă conține: nodul j , x_{ij} , r_{ij} , un pointer către elementul corespunzător lui i din lista de adiacență $A(j)$ și un pointer către următorul element din lista $A(i)$.
- lista are asociat un iterator pentru a-i înlesni parcurgerea.

Toate aceste liste sunt construite în $O(m)$, înaintea apelului procedurii **initialization**.

```

initialization;  $K \leftarrow \lceil \log_2 U \rceil$ ;  $\Delta \leftarrow 2^{K+1}$ ;
for ( $p = \overline{K, 0}$ ) do
    construiește  $L(p)$ ;  $\Delta \leftarrow \Delta/2$ ;
    while ( $L(p) \neq \emptyset$ ) do
        fie  $i$  primul nod din  $L(p)$ ;
        caută în  $A(i)$  primul arc admisibil sau până se ajunge la sfârșit;
        if ( $ij$  este arcul admisibil găsit) then
             $\delta \leftarrow \min(e_i, r_{ij}, \Delta - e_j)$ ;
             $e_i \leftarrow e_i - \delta$ ;  $e_j \leftarrow e_j + \delta$ ;
            "trimite"  $\delta$  unități de flux de la  $i$  la  $j$ ;
            if ( $e_i \leq \Delta/2$ ) then
                șterge  $i$  din  $L(p)$ ;
            if ( $e_j > \Delta/2$ ) then
                adaugă  $j$  la începutul listei  $L(p)$ ;
        else
            calculează  $d[i] = \min \{d[j] + 1 : ij \in A(i) \text{ și } r_{ij} > 0\}$ 
            re poziționează  $i$  în  $L(p)$ ;
            setează poziția curentă (a pointerului) la începutul listei  $A(i)$ ;
    
```

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Complexitatea timp a algoritmului este dominată de **pompările nesaturate** (ceea ce rămâne este de complexitate $\mathcal{O}(nm)$).

Lema 5

Numărul pompărilor nesaturate este cel mult $8n^2$ în fiecare etapă a scalării, astfel numărul total este $\mathcal{O}(n^2 \log U)$.

Demonstrație. Fie

$$F(p) = \sum_{i \in V, i \neq s, t} \frac{e_i \cdot d(i)}{2^p}.$$

La începutul etapei p , $F(p) < \sum_{i \in V} \frac{2^p \cdot (2n)}{2^p} = 2n^2$.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Dacă, în etapa p , se execută **relabel**(i), atunci nu există arce admisibile ij , și $d(i)$ crește cu $h \geq 1$ unități. $F(p)$ va crește cu cel mult h . Deoarece, $\forall i, d(i) < 2n$ urmează că $F(p)$ va crește (până la finalul etapei p) cel mult până la $4n^2$.

Dacă, în etapa p , se execută **push**(i), atunci aceasta trimite $\delta \geq 2^{p-1}$ pe arcul admisibil ij cu $r_{ij} > 0$ și $d(i) = d(j) + 1$. Astfel, după **push**, $F(p)$ va avea valoarea $F'(p) = F(p) - \frac{\delta \cdot d(i)}{2^p} + \frac{\delta \cdot d(j)}{2^p} = F(p) - \frac{\delta}{2^p} \leq F(p) - \frac{2^{p-1}}{2^p} = F(p) - 1/2$.

Această descreștere nu poate apărea de mai mult de $8n^2$ ori (deoarece $F(p)$ poate crește cel mult până la $4n^2$ și $F(p)$ este nenegativ). Evident, numărul pompărilor nesaturate este dominat de acest număr de descreșteri ale lui $F(p)$.

A. Determinarea unui cuplaj de cardinal maxim și a unei mulțimi stabile de cardinal maxim într-un graf bipartit.

Fie $G = (V_1, V_2; E)$ un graf bipartit cu n noduri și m muchii.

Considerăm rețeaua $R = (G_1, s, t, c)$, unde

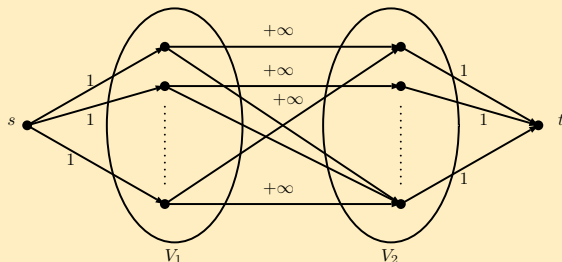
- $V(G_1) = \{s, t\} \cup V_1 \cup V_2$;
- $E(G_1) = E_1 \cup E_2 \cup E_3$, cu

$$E_1 = \{sv_1 : v_1 \in V_1\}, E_2 = \{v_2t : v_2 \in V_2\},$$

$$E_3 = \{v_1v_2 : v_1 \in V_1, v_2 \in V_2\},$$

- $c : E(G_1) \rightarrow \mathbb{N}$ definită prin

$$c(e) = \begin{cases} 1, & \text{dacă } e \in E_1 \cup E_2 \\ \infty, & \text{dacă } e \in E_3 \end{cases}$$



Dacă $x = (x_{ij})$ este un flux întreg în R , atunci mulțimea $\{ij : i \in V_1, j \in V_2 \text{ și } x_{ij} = 1\}$ corespunde unui cuplaj M^x în graful bipartit G , cu $|M^x| = v(x)$.

Reciproc, orice cuplaj $M \in \mathcal{M}_G$ corespunde unei mulțimi de arce neadiacente din G_1 ; dacă pe fiecare astfel de arc ij ($i \in V_1, j \in V_2$) considerăm $x_{ij}^M = 1$ și $x_{si}^M = x_{jt}^M = 1$, și adăugăm $x^M(e) = 0$ pe restul arcelor, atunci fluxul întreg x^M satisface $v(x^M) = |M|$.

Aplicații combinatoriale - Cuplaje în grafuri bipartite

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Astfel, dacă rezolvăm problema fluxului maxim în R (începând cu fluxul nul), atunci obținem în $\mathcal{O}(nm + n^2 \log n)$ un cuplaj de cardinal maxim în G .

Fie (S, T) secțiunea de capacitate minimă (obținută în $\mathcal{O}(m)$ dintr-un flux maxim determinat). Din Teorema de flux maxim- secțiune minimă, $c(S, T) = \nu(G)$.

Deoarece $\nu(G) < \infty$, luând $S_i = S \cap V_i$ și $T_i = T \cap V_i$ ($i = \overline{1, 2}$), avem $|T_1| + |S_2| = \nu(G)$ și $X = S_1 \cup T_2$ este o mulțime stabilă în G (pentru a avea $c(S, T) < \infty$). Mai mult, $|X| = |V_1 \setminus T_1| + |V_2 \setminus S_2| = n - \nu(G)$. Urmează că X este o mulțime stabilă de cardinal maxim, deoarece $n - \nu(G) = \alpha(G)$ (din teorema lui König).

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

B. Recunoașterea secvențelor digrafice

Considerăm următoarea problemă:

Date $(d_i^+)_{i=\overline{1,n}}$ și $(d_i^-)_{i=\overline{1,n}}$, există un digraf $G = (\{1, \dots, n\}, E)$ astfel încât $d_G^+(i) = d_i^+$ și $d_G^-(i) = d_i^-$, $\forall i = \overline{1, n}$?

Condiții evident necesare pentru un răspuns afirmativ sunt:

$$d_i^+ \in \mathbb{N}, 0 \leq d_i^+ \leq n - 1 \text{ și } d_i^- \in \mathbb{N}, 0 \leq d_i^- \leq n - 1, \forall i = \overline{1, n};$$

$$\sum_{i=1}^n d_i^+ = \sum_{i=1}^n d_i^- = m \text{ (unde } m = |E| \text{)}.$$

În aceste ipoteze considerăm rețeaua bipartită $R = (G_1, s, t, c)$.

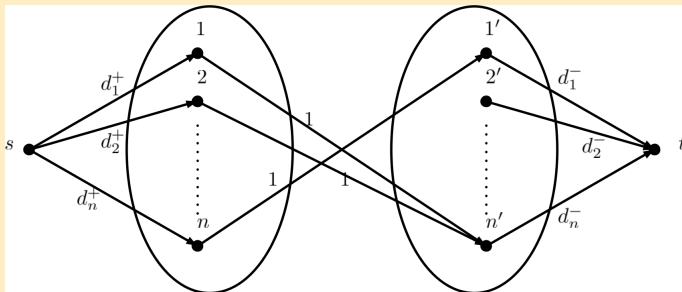
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Aplicații combinatoriale - Recunoașterea secvențelor digrafice

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

G_1 este obținut din graful complet bipartit $K_{n,n}$ cu bipartiția $(\{1, 2, \dots, n\}, \{1', 2', \dots, n'\})$, prin ștergerea mulțimii de muchii $\{11', 22', \dots, nn'\}$ și prin orientarea fiecărei muchii ij' ($\forall i \neq j \in \{1, 2, \dots, n\}$) de la i la j' , și prin adăugarea a două noi noduri s, t , și a tuturor arcelor $si, i \in \{1, 2, \dots, n\}$ și $j't, j \in \{1, 2, \dots, n\}$.

Funcția de capacitate: $c(si) = d_i^+, c(j't) = d_j^-, c(ij') = 1, \forall i, j = \overline{1, n}$.



C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Dacă în R există un flux întreg, x , de valoare maximă m , atunci din fiecare nod i vor pleca exact d_i^+ arce, ij' , pe care $x_{ij'} = 1$, și în fiecare nod j' vor intra exact d_i^- arce, ij' , pe care $x_{ij'} = 1$.

Digraful dorit, G , este construit luând $V(G) = \{1, 2, \dots, n\}$ și punând $ij \in E(G)$ dacă și numai dacă $x_{ij'} = 1$.

Reciproc, dacă G există, atunci inversând construcția anterioară vom obține un flux întreg în R de valoare m (deci de valoare maximă).

Urmează că, recunoașterea secvențelor digrafice (și construirea digrafului, în cazul unui răspuns afirmativ) poate fi făcută în $\mathcal{O}(nm + n^2 \log n) = \mathcal{O}(n^3)$.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Determinarea numărului de conexiune pe muchii a unui graf

Fie $G = (V, E)$ un graf. Pentru $s, t \in V$, $s \neq t$, notăm

- $p_e(s, t)$ = numărul maxim de drumuri disjuncte pe muchii de la s la t în G ,
- $c_e(s, t)$ = cardinalul minim al unei mulțimi de muchii astfel încât nu există drum de la s la t în graful obținut prin ștergerea ei din G .

Teoremă

$$p_e(s, t) = c_e(s, t).$$

Demonstrație. Fie G_1 digraful obținut din G prin înlocuirea fiecărei muchii printr-o pereche de arce simetrice. Fie $c : E(G_1) \rightarrow \mathbb{N}$ o funcție de capacitate definită prin $c(e) = 1$, $\forall e \in E(G_1)$.

Demonstrație (continuare). Fie x^0 un flux întreg de valoare maximă în $R = (G_1, s, t, c)$. Dacă există un circuit C în G_1 cu $x_{ij}^0 = 1$ pe toate arcele lui C , atunci putem pune fluxul 0 pe toate arcele lui C fără a modifica valoarea fluxului x_0 . Astfel, putem presupune că fluxul x^0 este aciclic și atunci x^0 poate fi scris ca o sumă de $v(x^0)$ fluxuri întregi x^k fiecare cu $v(x^k) = 1$.

Fiecare flux x^k corespunde unui drum de la s la t în G_1 (considerând arcele pe care fluxul nu este 0), care este un drum de la s la t și în graful G .

Urmează că $v(x^0) = p_e(s, t)$, deoarece orice mulțime de drumuri disjuncte pe muchii de la s la t în G generează un flux 0 – 1 în R de valoare egală cu numărul acestor drumuri.

Proof (continuare). Fie (S, T) o secțiune de capacitate minimă în R ; avem $c(S, T) = v(x^0)$, din Teorema flux maxim-secțiune minimă. Pe de altă parte, $c(S, T)$ este numărul de arce cu o extremitate în S și cealaltă în T (deoarece toate capacitățile arcelor sunt 1). Această mulțime de arce generează în G o mulțime de muchii de același cardinal astfel încât nu există vreun drum de la s la t în graful obținut prin ștergerea ei din G .

Astfel am obținut o mulțime de $c(S, T) = v(x^0) = p_e(s, t)$ muchii în G care deconectează pe s de t prin ștergerea lor din G . Urmează că $c_e(s, t) \leq p_e(s, t)$. Deoarece inegalitatea $c_e(s, t) \geq p_e(s, t)$ este evidentă, teorema este demonstrată. $\square$

Dacă G este un graf conex, $\lambda(G)$, valoarea maximă alui $p \in \mathbb{N}$ pentru care G este p -muchie-conex, este

$$\min_{s, t \in V(G), s \neq t} c_e(s, t) (*)$$

Urmează că, pentru a calcula $\lambda(G)$, este necesar să rezolvăm $n(n-1)/2$ probleme de flux maxim descrise mai sus. Acest număr poate fi redus dacăe observăm că, pentru o pereche fixată (s, t) , dacă (S, T) este o secțiune de capacitate minimă, atunci

$$\forall v \in S \text{ și } \forall w \in T \quad c_e(v, w) \leq c(S, T) (**)$$

În particular, dacă (s, t) este perechea pentru care minimul din $(*)$ se atinge, avem egalitate în $(**)$.

Dacă fixăm un nod $s_0 \in V$ și rezolvăm cele $n-1$ probleme de flux maxim luând $t_0 \in V \setminus \{s_0\}$ vom obține o pereche (s_0, t_0) cu $c(s_0, t_0) = \lambda(G)$ (t_0 nu va fi în aceeași clasă a bipartiției cu s_0 în (S, T)).

Concluzie: $\lambda(G)$ poate fi găsit în $\mathcal{O}(n \cdot (nm + n^2 c)) = \mathcal{O}(n^2 m)$.

D. Determinarea numărului de conexiune pe noduri a unui graf.

Fie $G = (V, E)$ un graf. $\forall s, t \in V, s \neq t$, dacă notăm

- $p(s, t)$ = numărul maxim de drumuri intern disjuncte (pe noduri) de la s la t în G ,
- $c(s, t)$ = cardinalul minim al unei mulțimi s - t -separatoare de noduri din G ,

atunci, Din teorema lui Menger, avem

$$p(s, t) = c(s, t) (***)$$

În plus, numărul de conexiune pe noduri, $k(G)$, al grafului G (valoarea maximă a lui $p \in \mathbb{N}$ pentru care G este p -conex) este

$$k(G) = \begin{cases} n - 1, & \text{dacă } G = K_n \\ \min_{s, t \in V(G), s \neq t} c(s, t), & \text{dacă } G \neq K_n \end{cases} (****)$$

Aplicații combinatoriale - Conexiunea pe noduri

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

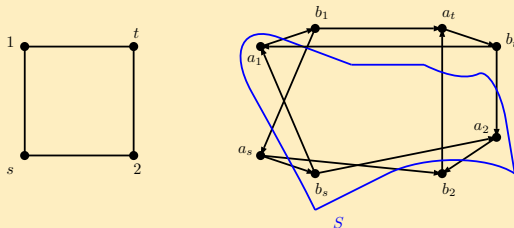
Arătăm că egalitatea ($*$ $*$ $*$) provine din Teorema flux maxim-secțiune minimă, aplicată unei rețele corespunzătoare.

Fie $G_1 = (V(G_1), E(G_1))$ digraful construit pornind de la G astfel:

- $\forall v \in V$, adăugăm $a_v, b_v \in V(G_1)$ și $a_v b_v \in E(G_1)$;
- $\forall vw \in E$, adăugăm $b_v a_w, b_w a_v \in E(G_1)$.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Exemplu



Mai definim $c : E(G_1) \rightarrow \mathbb{N}$ prin

$$c(e) = \begin{cases} 1, & \text{dacă } e = a_v b_v \\ \infty, & \text{altfel} \end{cases}$$

Considerăm rețeaua $R = (G_1, b_s, a_t, c)$.

Fie x^0 un flux întreg în R de valoare maximă. În nodurile $b_v (v \in V)$ intră exact câte un arc de capacitate 1 și din nodurile $a_v (v \in V)$ pleacă exact câte un arc de capacitate 1.

Urmează că (din legea de conservare a fluxului) că $x_{ij}^0 \in \{0, 1\}$, $\forall ij \in E(G_1)$. Astfel x^0 poate fi descompus în $v(x^0)$ fluxuri x^k , fiecare de valoare 1, cu proprietatea că arcele pe care x_k este nenul corespund la $v(x^0)$ drumuri intern disjuncte din G .

Pe de altă parte, din orice mulțime de p st -drumuri intern disjuncte în G , putem construi p $b_s a_t$ -drumuri intern disjuncte în G_1 , pe care se poate transporta o singură unitate de flux. Urmează că $v(x^0) = p(s, t)$.

Fie (S, T) o secțiune de capacitate minimă în R astfel încât $v(x^0) = c(S, T)$. Deoarece $v(x^0) < \infty$, urmează că $\forall i \in S, \forall j \in T$ cu $ij \in E(G_1)$; avem $c(ij) < \infty$, deci $c(ij) = 1$, adică $\exists u \in V$ astfel încât $i = a_u$ și $j = b_u$.

Astfel, secțiunea (S, T) corespunde unei mulțimi de noduri $A_0 \subseteq V$ astfel încât $c(S, T) = |A_0|$ și A_0 este o mulțime st -separatoare.

Pe de altă parte, pentru orice mulțime st -separatoare, A , avem $|A| \geq p(s, t) = v(x^0)$. Deci

$$c(s, t) = |A_0| = c(S, T) = v(x^0) = p(s, t).$$

Demonstrația de mai sus arată că, pentru a determina $k(G)$ este suficient să determinăm minimul în $(***)$ rezolvând $|E(\overline{G})|$ probleme de flux maxim, unde $\overline{G}$ este complementul lui G .

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Avem astfel un algoritm cu complexitatea timp

$$\mathcal{O}\left(\left(\frac{n(n-1)}{2} - m\right)(nm + n^2 \log n)\right).$$

O observație simplă ne oferă un algoritm mai eficient. Evident,

$$k(G) \leq \min_{v \in V} d_G(v) = \frac{1}{n} \left(n \cdot \min_{v \in V} d_G(v) \right) \leq \frac{1}{n} \sum_{v \in V} d_G(v) = \frac{2m}{n}.$$

Dacă A_0 este o mulțime separatoare în G cu $|A_0| = k(G)$, atunci $G \setminus A_0$ nu este conex și există o partiție a lui $V \setminus A_0$ (V' , V'') astfel încât $\forall v' \in V'$ și $\forall v'' \in V''$ avem $p(v', v'') = k(G)$.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Urmează că rezolvând o problemă de flux maxim cu $s_0 \in V'$ și $t_0 \in V''$ obținem că $p(s_0, t_0) = \text{valoarea fluxului maxim} = k(G)$.

Putem determina o astfel de pereche după cum urmează: fie $l = \left\lceil \frac{2m}{n} \right\rceil + 1$, alegem l noduri arbitrare din $V(G)$, și pentru fiecare astfel de nod, v , rezolvăm toate problemele de flux maxim $p(v, w)$, cu $vw \notin E$. Numărul acestor probleme este $\mathcal{O}(nl) = \mathcal{O}\left(n\left(\left\lceil \frac{2m}{n} \right\rceil + 1\right)\right) = \mathcal{O}(m)$.

Astfel complexitatea timp a determinării lui $k(G)$ este $\mathcal{O}(m(nm + n^2 \log n))$.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții pentru seminarul din săptămâna viitoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 1. Arătați că, utilizând un algoritm de flux maxim (într-o anumită rețea), se poate găsi, într-o matrice 0 – 1, o mulțime de cardinal maxim de elemente egale cu 0, în care oricare două elemente nu se află pe aceeași linie sau pe aceeași coloană.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Exercițiul 2. Fie S și T mulțimi nevide, disjuncte și finite. Se dă o funcție $a : S \cup T \rightarrow \mathbb{N}$. Să se decidă dacă există un graf bipartit $G = (S, T; E)$ astfel încât $d_G(v) = a(v)$, pentru orice $v \in S \cup T$; dacă răspunsul este afirmativ trebuie returnate muchiile lui G (S și T sunt clasele bipartiției lui G). Arătați că această problemă poate fi rezolvată în timp polinomial ca o problemă de flux maxim într-o anumită rețea.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 3. Fiecare student dintr-o mulțime S de cardinal $n > 0$ subscrie la o submulțime de 4 cursuri opționale dintr-o mulțime C de cardinal $k > 4$. Descrieți un algoritm (de complexitate timp polinomială) care să determine (dacă există) o alocare a studenților către cursurile opțional din C astfel încât fiecare student să fie asignat la exact 3 cursuri (din cele 4 la care a scris) și fiecare curs să aibă asignați cel mult $\lceil n/k \rceil$ studenți.

Exercițiul 4.

- (a) Adevărat sau fals? Într-o rețea $R = (G, s, t, c)$ cu capacități distincte există un singur flux maxim. De ce?
- (b) Descrieți și demonstrați corectitudinea unui algoritm de complexitate (timp) polinomială care să decidă dacă într-o rețea dată există un singur flux maxim.

Exerciții pentru seminarul din săptămâna viitoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 5. O companie IT are n angajați $P_1, P_2, \dots, P_n$ care trebuie să lucreze la m proiecte $L_1, L_2, \dots, L_m$. Pentru orice angajat P_i avem o listă $\mathcal{L}_i$ a proiectelor la care poate lucra și s_i numărul de proiecte din $\mathcal{L}_i$ pe care le poate termina într-o săptămână ($s_i \leq |\mathcal{L}_i|$). Fiecare proiect va fi alocat unui singur angajat.

Cum se poate determina numărul minim de săptămâni necesare terminării tuturor proiectelor folosind fluxuri în rețele.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 6. Planul de evacuare de urgență al unei clădiri este descris ca un grid $n \times n$; frontierele celulelor sunt rutele de evacuare către exteriorul clădirii (grid-ului). O instanță a **problemei evacuării** conține dimensiunea n a grid-ului și m puncte de plecare (colțurile celulelor).

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 6 (continuare). Instanța primește un răspuns afirmativ dacă există m drumuri disjuncte către frontiera grid-ului plecând din cele m puncte de mai sus. Dacă drumurile acestea nu există, atunci instanța primește un răspun negativ.

Determinați o reprezentare a acestei **problem a evacuării** ca o problemă de flux într-o rețea. Descrieți un algoritm eficient pentru a recunoaște instanțele pozitive ale problemei (care este complexitatea timp a algoritmului?).

Exercițiul 7. Fie $G = (V, E)$ un graf cu n noduri $\{v_1, v_2, \dots, v_n\}$ și $c : E \rightarrow \mathbb{R}_+$ o funcție de capacitate pe muchiile lui G . O **tăietură** în G este o bipartiție (S, T) a lui V . Capacitatea unei tăieturi (S, T) este
$$c(S, T) = \sum_{e \in E, |e \cap S|=1} c(e).$$

Exerciții pentru seminarul din săptămâna viitoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 7 (continuare). O tăietură minimă în G este o tăietură (S_0, T_0) astfel încât

$$c(S_0, T_0) = \min_{(S, T) \text{ tăietură în } G} c(S, T)$$

- (a) Arătați că se poate determina o tăietură minimă în timp polinomial rezolvând un număr polinomial de probleme de flux maxim în anumite rețele.
- (b) Pentru $G = C_n$ (circuit indus de ordin $n \geq 3$) cu toate capacitățile 1, arătați că există $\frac{n(n-1)}{2}$ tăieturi minime.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 8. Fie $G = (V; E)$ un digraf și $w : V \rightarrow \mathbb{R}$ astfel ca $w(V) \cap \mathbb{R}_+$ și $w(V) \cap \mathbb{R}_- \neq \emptyset$. O submulțime $A \subseteq V$ se numește *izolată* în G dacă nu există nici un arc care iese din A . *Ponderea* unei submulțimi $A \subseteq V$ este $w(A) = \sum_{v \in A} w(v)$. Descrieți un algoritm de complexitate polinomială bazat pe un algoritm de flux maxim într-o anumită rețea care să determine o submulțime izolată de pondere maximă în G .

Exercițiul 9. La Departamentul de Informatică există p studenți ($S = \{S_1, S_2, \dots, S_p\}$) care doresc să aprobe cu o diplomă de licență și k profesori ($\mathbb{P} = \{P_1, P_2, \dots, P_k\}$). Lucrările de licență ale studenților sunt evaluate de echipe formate din câte r profesori.

Pentru subiectul unei lucrări de licență un profesor poate specializat sau nu; se cunoaște mulțimea, $\mathbb{P}_i \subsetneq \mathbb{P}$ ($\mathbb{P}_i \neq \emptyset$), a profesorilor competenți în a judeca lucrarea de licență a studentului S_i , pentru orice i . Fiecare profesor P_l poate participa în cel mult n_l astfel de echipe de evaluare.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 9 (cont.) Fiecare student trebuie să-și prezinte lucrarea unei echipe de $r(\leq k)$ profesori, $a(\leq r)$ fiind specializați în proiectul respectiv, iar $(r - a)$ nu.

- (a) Descrieți un model cu o rețea de transport pentru a organiza ca mai sus echipele de evaluare formate din profesori (fiecare profesor trebuie asignat unei mulțimi de lucrări de licență).
- (b) Caracterizați existența unei soluții pentru această problemă în termenii existenței unui anumit flux maxim în rețeaua de mai sus. (Caracterizarea trebuie demonstrată!)
- (c) Care este complexitatea timp necesară pentru a decide dacă există soluții?

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 1. Soluție.

- Mai întâi se reduce problema la determinarea unui cuplaj maxim într-un anume graf bipartit.
- Fie $A = (a_{ij})_{\substack{1 \leq i \leq n \\ 1 \leq j \leq m}}$, definim $G = (S, T; E)$ astfel

$$S = \{v_1, v_2, \dots, v_n\}, T = \{u_1, u_2, \dots, u_m\}, E = \{v_i u_j : a_{ij} = 0\}.$$

- Fie $M \subseteq E(G)$ și $P = \{a_{ij} : v_i u_j \in M\}$; $M \in \mathcal{M}_G$ dacă și numai dacă orice două elemente din P aparțin la linii și coloane diferite (de ce?).
- Cum $|P| = |M|$, un cuplaj maxim corespunde unei mulțimi de cardinal maxim cu proprietățile de mai sus,
- Putem folosi un algoritm de flux maxim pentru a determina un cuplaj maxim într-un graf bipartit (cum?).

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 2. Soluție.

- Putem reformula problema ca una de recunoaștere a unei secvențe digrafice.
- Există un digraf G' cu $V(G') = S \cup T$, și $d_{G'}^-(v) = d^-(v)$ și $d_{G'}^+(v) = d^+(v)$, pentru orice $v \in V$, unde

$$d_{G'}^-(v) = \begin{cases} a(v), & v \in T \\ 0, & v \in S \end{cases}, \quad d_{G'}^+(v) = \begin{cases} a(v), & v \in S \\ 0, & v \in T \end{cases} ?$$

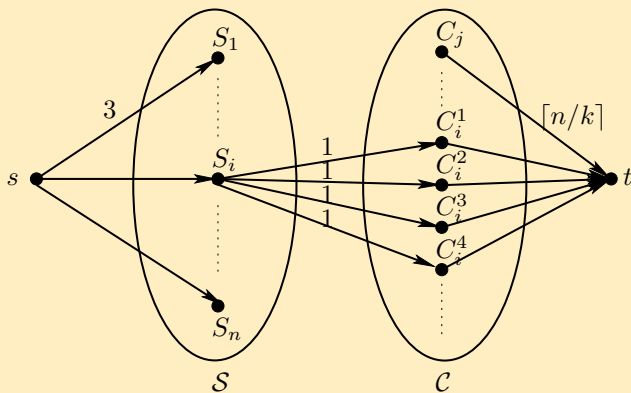
- Apoi se poate utiliza un algoritm de flux maxim într-o anumită rețea (**cum?**).

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 3. Soluție.



Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

- Presupunem că mulțimea studenților este $S = \{S_1, S_2, \dots, S_n\}$, cea a cursurilor $C = \{C_1, C_2, \dots, C_k\}$ iar fiecare student S_i a ales $\{C_i^1, C_i^2, C_i^3, C_i^4\} \subset C$, for $i = \overline{1, n}$.
- Definim o rețea de transport: $R = (G, s, t, c)$, unde $c : E(G) \rightarrow \mathbb{R}_+$ și

$$V = V(G) = \{s, t\} \cup \Sigma \cup C$$

$$E = E(G) = \{sS_i : i = \overline{1, n}\} \cup \{S_iC_i^l : i = \overline{1, n}, l = \overline{1, 4}\} \\ \cup \{C_jt : j = \overline{1, k}\}$$

$$c(sS_i) = 3, \forall i = \overline{1, n}; \quad c(S_iC_i^l) = 1, \forall i = \overline{1, n}, l = \overline{1, 4};$$

$$c(C_jt) = \lceil n/k \rceil, \forall j = \overline{1, k}.$$

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

- Există o alocare a cursurilor/studenților cu proprietățile cerute dacă și numai dacă valoarea maximă a unui flux în R este $3n$ (altfel spus toate arcele care părăsesc nodul s sunt saturate - **de ce?**).
- “ $\implies$ ” Dacă avem o alocare astfel ca orice student S_i participă la 3 cursuri C_{i1}, C_{i2}, C_{i3} , iar fiecare curs C_j adună $m_j \leq \lceil n/k \rceil$ studenți, atunci fie $x : E \rightarrow \mathbb{R}_+$:

$$x(sS_i) = 3, \forall i = \overline{1, n}; \quad x(S_i C_{il}) = 1, \forall i = \overline{1, n}, l = \overline{1, 3};$$

$$x(C_j t) = m_j, \forall j = \overline{1, k}.$$

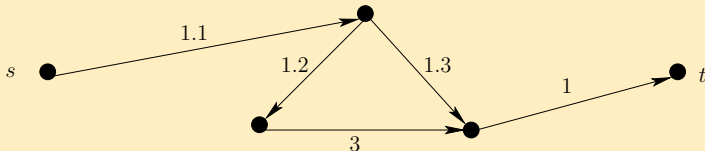
- Evident x este un flux maxim în R de valoare $3n$ (**de ce?**).
- “ $\impliedby$ ” ... **cum?**

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

Exercițiul 4. Soluție.

(a)



(b) Fie x un flux de valoare maximă în rețeaua R ; arătăm că x este singurul flux de valoare maximă dacă și numai dacă nu există A -drumuri închise relativ la x .

- " $\implies$ " Să presupunem că există un A -drum P închis relativ la x ($\delta(P) > 0$). Definim următoarea funcție $y : E(G) \rightarrow \mathbb{R}_+$ prin

$$y(e) = \begin{cases} x(e), & \text{if } e \notin E(P), \\ x(e) - \delta(P), & \text{if } e \in E(P), \text{ is a direct arc,} \\ x(e) + \delta(P), & \text{if } e \in E(P), \text{ is an inverse arc,} \end{cases}$$

$y (\neq x)$ este deasemeni flux maxim în R (de ce?) - contradicție.

- " $\impliedby$ " Să presupunem prin reducere la absurd că există două fluxuri maxime distincte în R , x și y . Următorul algoritm oferă un A -drum închis în R :

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

```
for( $v \in V(G)$ ) {  
    mark[v], before[v]  $\leftarrow$  0;}  
let  $e = uv \in E(G)$  s. t.  $x(uv) \neq y(uv)$ ;  
mark[u], mark[v]  $\leftarrow$  1;  
 $z \leftarrow v$ ; before[v]  $\leftarrow u$ ; orientation  $\leftarrow$  direct;  
while(true) {  
    if(orientation = direct) {  
        if( $\exists e' = wz \in E(G)$  s. t.  $sign(e) \neq sign(e')$ )  
            orientation  $\leftarrow$  inverse;  
        else let  $e' = zw \in E(G)$  s. t.  $sign(e) = sign(e')$ ;  
        if(mark[w] = 1)  
            return;  
        mark[w]  $\leftarrow$  1; before[w]  $\leftarrow z$ ;  $z \leftarrow w$ ,  $e \leftarrow e'$ ;  
    }  
}
```

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

```
if(orientation = inverse) {  
    if( $\exists e' = zw \in E(G)$  s. t.  $\text{sign}(e) \neq \text{sign}(e')$ )  
        orientation  $\leftarrow$  direct;  
    else let  $\exists e' = wz \in E(G)$  s. t.  $\text{sign}(e) = \text{sign}(e')$ ;  
    if(mark[w] = 1)  
        return;  
    mark[w]  $\leftarrow$  1; before[w]  $\leftarrow$  z; z  $\leftarrow$  w, e  $\leftarrow$  e';  
}  
} // for while
```

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

- Existența arcului e' este garantată de relațiile (de ce?):

$$\sum_{ij \in E(G)} x(ij) - \sum_{ji \in E(G)} x(ji) = \sum_{ij \in E(G)} y(ij) - \sum_{ji \in E(G)} y(ji), \forall i \in V(G),$$

- Buclo *while* de mai sus se încheie prin detectarea un nod deja marcat, i.e. când găsim un drum închis, P , care are proprietatea:

$$x(e) \begin{cases} > 0, & \text{dacă } e \text{ este un arc direct,} \\ < c(e), & \text{dacă } e \text{ este un arc invers} \end{cases}.$$

- Dacă P nu este un A -drum închis relativ la x , atunci parcurgându-l invers, obținem un A -drum închis relativ la x - ceea ce e o contradicție (de ce?).
- Putem verifica în $\mathcal{O}(n + m)$ dacă există un A -drum închis relativ la un flux maxim x folosind o strategie *bfs* asemănătoare celei din algoritmul Edmonds-Karp.

Exercițiul 5. Soluție.

- Fie $R_k = (G, s, t, c_k)$ o rețea (pentru k săptămâni) unde

$$V(G) = \{s, t\} \cup \{P_1, P_2, \dots, P_n, L_1, L_2, \dots, L_m\},$$

$$E(G) = \{sP_i : 1 \leq i \leq n\} \cup \{L_j t : 1 \leq j \leq m\} \cup$$

$$\{P_i L_j : L_j \in \mathcal{L}_i, 1 \leq i \leq n\},$$

$$c_k(sP_i) = ks_i, i = \overline{1, n}, c_k(P_i L_j) = \infty, \text{ for } L_j \in \mathcal{L}_i,$$

$$c_k(L_j t) = 1, j = \overline{1, m}$$

- Valoarea unui flux maxim este cel mult $\sum_{j=1}^m c_k(L_j t) = m$ (de ce?),

adică valoarea necesară pentru incheia toate proiectele. Astfel, numărul minim de săptămâni este

$$\min\{k : v(\varphi) = m, \varphi \text{ flux maxim in } R_k\} \text{ (de ce?)}$$

Exerciții rezolvate (parțial)

- Acest minim poate fi găsit folosind o procedură de căutare binară (notăm cu $v(R_k)$ valoarea maximă a unui flux în R_k):

$min \leftarrow 0$;

$max \leftarrow ?$;

while ($min < max - 1$) {

$current \leftarrow ?$;

 if ($v(R_{current}) < m$)

$min \leftarrow current$;

 else

$max \leftarrow current$;

 }

if ($v(R_{min}) == m$)

 return min ;

else

 return max ;

Exercițiul 7. Soluție.

(a) Fără a restrânge generalitatea putem presupune că $G \equiv K_n$ (adăugăm muchiile lipsă și punem capacități nule).

- Fie s și t două noduri noi ($s, t \notin V$) iar H următorul digraf

$$V(H) = V(G) \cup \{s, t\},$$

$$E(H) = \{uv, vu : uv \in E(G)\} \cup \{su, ut : u \in V(G)\}.$$

- Fie $\alpha > |E| \cdot \max_{e \in E} c(e) + 1$ mai mare decât capacitatea oricărei tăieturi din G).
- Definim rețeaua $R_{ij} = (H, s, t, c_{ij})$, unde

$$c_{ij}(uv) = c(uv), \forall uv \in E(G),$$

$$c_{ij}(sv) = c_{ij}(vt) = \alpha, \forall v \in V(G) \setminus \{v_i, v_j\} \text{ și } c(sv_i) = c(v_j t) = 0$$

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

- Fie (S', T') o secțiune de capacitate minimă în rețeaua R_{ij} .
- Notăm $S = S' \setminus \{s\}$ și $T = T' \setminus \{t\}$, atunci S și T sunt amândouă vide:
- Presupunem prin reducere la absurd că $S = \emptyset$, atunci $c_{ij}(S', T') = (n-1)\alpha$, but $c_{ij}(S' \cup \{v_j\}, T' \setminus \{v_j\}) = (n-2)\alpha + c(\{v_j\}, V \setminus \{v_j\}) < c(S', T')$ - contradicție (de ce?).
- Similar se poate arăta că $T \neq \emptyset$.
- $c_{ij}(S', T') = c(S, T) + (n-2)\alpha + a_{ij}(S, T)$ unde

$$a_{ij}(S, T) = \begin{cases} 2\alpha, & \text{if } |S \cap \{v_j\}| + |T \cap \{v_i\}| = 0, \\ \alpha, & \text{if } |S \cap \{v_j\}| + |T \cap \{v_i\}| = 1, \\ 0, & \text{if } |S \cap \{v_j\}| + |T \cap \{v_i\}| = 2. \end{cases}$$

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

- Deoarece $\alpha > 0$ vom avea $c_{ij}(S', T') = c(S, T) + (n - 2)\alpha$, astfel $v_j \in S$, și $v_i \in T$.
- Fie (S_0, T_0) o tăietură de capacitate minimă în G ; se poate arăta că

$$(n-2)\alpha + c(S_0, T_0) = \min_{1 \leq i \neq j \leq n} \min_{(S', T') \text{ secțiune în } R_{ij}} c_{ij}(S', T') \text{ (de ce?)}$$

- Astfel, pentru a determina o tăietură de capacitate minimă în G trebuie rezolvate $\frac{n(n-1)}{2}$ probleme de flux maxim (de ce?).

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 9. Soluție.

(a) Considerăm următoarea rețea, $R = (G, s, t, c)$, unde

$$V(G) = S \cup \mathcal{P} \cup \{s, t\},$$

$$E(G) = \{sS_i : i = \overline{1, p}\} \cup \{S_i C_i : i = \overline{1, p}\} \cup \{S_i N_i : i = \overline{1, p}\} \cup \\ \cup \{C_i P_l : P_l \in \mathcal{P}_i\} \cup$$

$$\cup \{N_i P_l : P_l \in \mathcal{P} \setminus \mathcal{P}_i\} \cup \{P_l t : l = \overline{1, k}\},$$

$$c(sS_i) = r, c(S_i C_i) = a, c(S_i N_i) = r - a,$$

$$c(C_i P_j) = (N_i P_j) = 1, c(P_l t) = n_l$$

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

(b) Există o soluție a problemei dacă și numai dacă fluxul maxim din această rețea are valoarea $p \cdot r$ (de ce?).

$$(c) |G| = 3p + k + 2, |E(G)| = 3p + \sum_{i=1}^p |\mathcal{P}_i| + \sum_{i=1}^p |\mathcal{P} \setminus \mathcal{P}_i| + k = pk + 3p + k.$$

Capacitatea maximă a unei rețele în R este

$$U = \max \{r, \max_{1 \leq l \leq k} n_l\} \leq \max \{r, p\} \leq \max \{k, p\}.$$

- Complexitatea algoritmului Edmonds-Karp este: $\mathcal{O}((p+k) \cdot p^2 \cdot k^2)$.
- Complexitatea algoritmului Ahuja-Orlin: ?
- Complexitatea algoritmului Ford-Fulkerson: ?

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *